

LAPORAN JOBSHEET 4
PRAKTIKUM PEMROGRAMAN VISUAL

WIDGET LANJUTAN PYSIDE6

Untuk Memenuhi Salah Satu Tugas
Mata Kuliah Praktikum Pemrograman Visual
Dosen Pengampu: Safri Adam, S.Kom., M.Kom



Disusun Oleh:

Rafi Achmad Arlino

3202416039

PROGRAM STUDI D3 TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI PONTIANAK
2022

KATA PENGANTAR

Puji dan syukur kami panjatkan kepada Allah SWT atas rahmat dan karunia-nya sehingga laporan yang berjudul “**WIDGET LANJUTAN PYSIDE6**” dapat terselesaikan dengan baik. Laporan ini merupakan salah satu tugas yang diberikan oleh dosen pengampu mata kuliah Praktikum Pemrograman Visual kepada mahasiswa Program Studi D3 Teknik Informatika Jurusan Teknik Elektro sebagai salah satu bagian dari komponen penilaian akademis.

Laporan ini membahas terkait tipe data, variabel, dan konstanta. Demikian Laporan ini saya buat, semoga bermanfaat.

Pontianak, 6 Oktober 2025

Penyusun,

(Rafi Achmad Arlino)

JOBSHEET 1
WIDGET LANJUTAN PYSIDE6

Percobaan 1 : QSpinBox dan QDoubleSpinBox

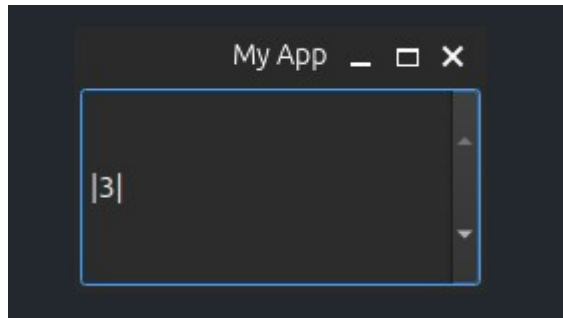
Source code :



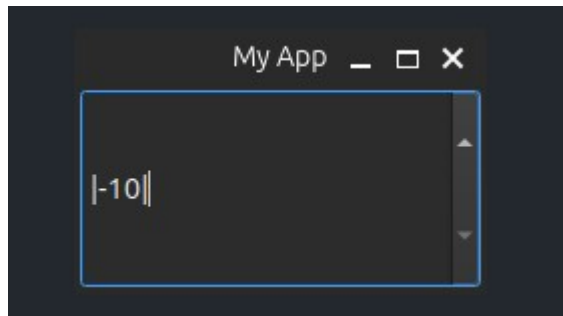
```
1  from PySide6.QtWidgets import (
2      QApplication,
3      QMainWindow,
4      QSpinBox,
5      QDoubleSpinBox,
6  )
7
8  import sys
9
10 class MainWindow(QMainWindow):
11     def __init__(self):
12         super().__init__()
13
14         self.setWindowTitle("My App")
15
16         widget = QSpinBox()
17
18         widget.setMinimum(-10)
19         widget.setMaximum(3)
20
21         widget.setPrefix("|")
22         widget.setSuffix("|")
23         widget.setSingleStep(1)
24         widget.valueChanged.connect(self.value_changed)
25         widget.textChanged.connect(
26             self.value_changed_str
27         )
28
29         self.setCentralWidget(widget)
30
31     def value_changed(self, i):
32         print(i)
33
34     def value_changed_str(self, s):
35         print(s)
36
37 app = QApplication(sys.argv)
38 window = MainWindow()
39 window.show()
40 app.exec()
41
```

Output :

SetMinimum :



SetMaximum :



Penjelasan :

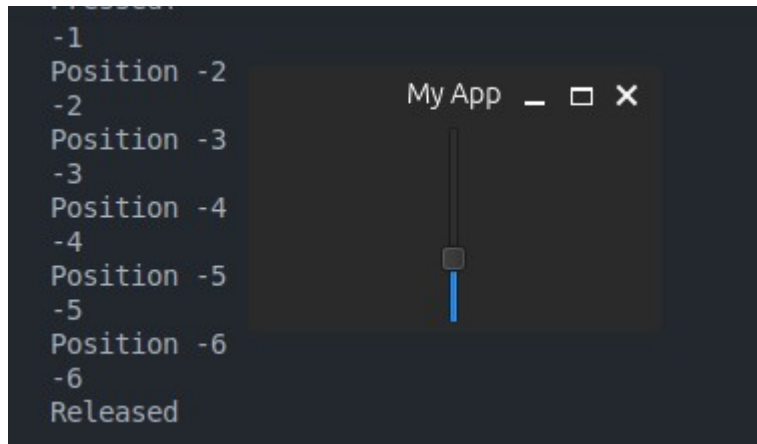
Ini merupakan percobaan dari **QSpinBox** dan **QDoubleSpinBox**. QSpinBox merupakan widget dengan anak panah dengan panah kecil untuk menambah atau mengurangi nilai. Qspinbox mendukung bilangan bulat sedangkan QDoubleSpinBox mendukung bilangan float.

Percobaan 2 : QSlider

Source code :

```
1 from PySide6.QtWidgets import (
2     QApplication,
3     QMainWindow,
4     QSpinBox,
5     QDoubleSpinBox,
6     QSlider
7 )
8
9 import sys
10
11 class MainWindow(QMainWindow):
12     def __init__(self):
13         super().__init__()
14
15         self.setWindowTitle("My App")
16
17         widget = QSlider()
18
19         widget.setMinimum(-10)
20         widget.setMaximum(3)
21
22         widget.setSingleStep(1)
23
24         widget.valueChanged.connect(self.value_changed)
25         widget.sliderMoved.connect(self.slider_position)
26         widget.sliderPressed.connect(self.slider_pressed)
27         widget.sliderReleased.connect(self.slider_released)
28
29         self.setCentralWidget(widget)
30
31     def value_changed(self, i):
32         print(i)
33
34     def slider_position(self, p):
35         print("Position", p)
36
37     def slider_pressed(self):
38         print("Pressed!")
39
40     def slider_released(self):
41         print("Released")
42
43 app = QApplication(sys.argv)
44 window = MainWindow()
45 window.show()
46 app.exec()
47
```

Output :



Penjelasan : QSlider merupakan widget slidebar yang fungsi internal nya juga mirip seperti spinbox. jika spinbox menampilkan nilai dengan sebuah tombol panah, slider menunjukkan nilai dengan sebuah bar yang dapat digeser geser layaknya sebuah bar volume. widget ini menurut saya lebih interaktif dan dapat dimodifikasi lebih lanjut. pengguna dapat menggunakan feeling agar mengatur sebuah nilai. setiap pergeseran slider dapat dilihat di terminal. seperti posisi slider di nilai -2 dan release artinya si slider sudah tidak ditekan dan digeser atau sudah lepas dari kursor

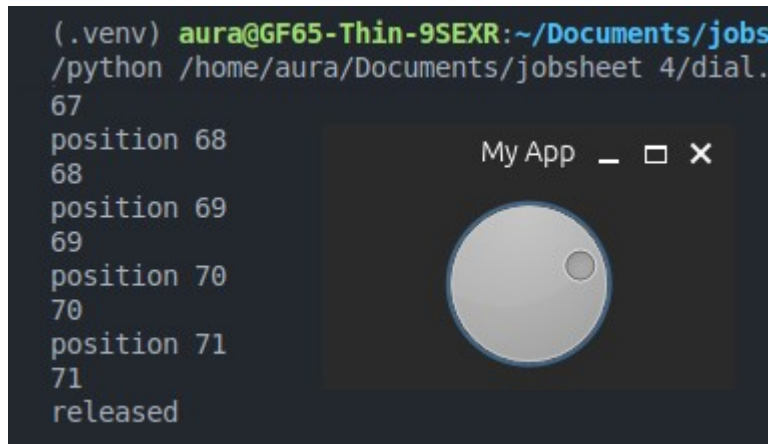
Percobaan 3 : Qdial

Source code :



```
1 import sys
2 from PySide6.QtWidgets import (
3     QDial,
4     QApplication,
5     QMainWindow,
6 )
7
8 class MainWindow(QMainWindow):
9     def __init__(self):
10         super().__init__()
11
12         self.setWindowTitle("My App")
13
14         Widget = QDial()
15         Widget.setRange(-10, 100)
16         Widget.setSingleStep(1)
17
18         Widget.valueChanged.connect(self.value_changed)
19         Widget.sliderMoved.connect(self.slider_position)
20         Widget.sliderPressed.connect(self.slider_pressed)
21         Widget.sliderReleased.connect(self.slider_released)
22
23         self.setCentralWidget(Widget)
24
25     def value_changed(self, i):
26         print(i)
27
28     def slider_position(self, p):
29         print("position", p)
30
31     def slider_pressed(self):
32         print("pressed")
33
34     def slider_released(self):
35         print("released")
36
37 app = QApplication(sys.argv)
38 window = MainWindow()
39 window.show()
40 app.exec()
```

Output :



Penjelasan :

Qdial merupakan widget seperti pemutar volume pada radio. fungsi nya juga sama seperti slider juga tetapi dengan tampilan yang lebih interaktif. namun secara aspek UI ini lebih menyusahkan user dibanding kan metode pengatur nilai lainnya.

Project 1 : Aplikasi pemutar musik sederhana

Source Code :

```
from PySide6.QtWidgets import (
    QApplication,
    QVBoxLayout,
    QWidget,
    QPushButton,
    QFileDialog,
    QSlider,
    QLabel,
)
from PySide6.QtCore import Qt, QTimer

from mutagen.mp3 import MP3
from mutagen.wave import WAVE
from mutagen.oggvorbis import OggVorbis

import pygame
import sys

class MusicPlayer(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Pemutar Musik Sederhana")
```

```

self.setGeometry(300, 200, 400, 300)

pygame.mixer.init()

self.layout = QVBoxLayout()

# Tombol
self.load_button = QPushButton("Muat Musik")
self.load_button.clicked.connect(self.load_music)
self.layout.addWidget(self.load_button)

self.play_button = QPushButton("Putar")
self.play_button.clicked.connect(self.play_music)
self.layout.addWidget(self.play_button)

self.stop_button = QPushButton("Stop Musik")
self.stop_button.clicked.connect(self.stop_music)
self.layout.addWidget(self.stop_button)

# Volume slider
self.volume_slider = QSlider(Qt.Horizontal)
self.volume_slider.setRange(0, 100)
self.volume_slider.setValue(30)
self.volume_slider.valueChanged.connect(self.set_volume
)

self.layout.addWidget(QLabel("Volume"))
self.layout.addWidget(self.volume_slider)

# Durasi slider
self.position_slider = QSlider(Qt.Horizontal)
self.position_slider.setEnabled(False)
self.position_slider.sliderReleased.connect(self.set_po
sition)

self.layout.addWidget(QLabel("Durasi"))
self.layout.addWidget(self.position_slider)

self.setLayout(self.layout)

# Variabel musik
self.music_file = None
self.duration = 0

```

```

        # Timer untuk update slider
        self.timer = QTimer()
        self.timer.timeout.connect(self.update_position)

    def load_music(self):
        file_dialog = QFileDialog(self)
        music_file, _ = file_dialog.getOpenFileName(
            self, "Pilih file Musik", "", "Audio Files (*.mp3
*.wav *.ogg)"
        )

        if music_file:
            self.music_file = music_file
            pygame.mixer.music.load(self.music_file)

            # Ambil durasi dengan mutagen
            if music_file.endswith(".mp3"):
                audio = MP3(music_file)
                self.duration = int(audio.info.length)
            elif music_file.endswith(".wav"):
                audio = WAVE(music_file)
                self.duration = int(audio.info.length)
            elif music_file.endswith(".ogg"):
                audio = OggVorbis(music_file)
                self.duration = int(audio.info.length)
            else:
                self.duration = 0

            self.position_slider.setRange(0, self.duration)
            self.position_slider.setEnabled(True)

    def play_music(self):
        if self.music_file:
            pygame.mixer.music.play()
            self.timer.start(1000) # update setiap detik
        else:
            print("Silahkan muat file musik terlebih dahulu!")

    def stop_music(self):
        pygame.mixer.music.stop()

```

```

        self.timer.stop()

    def set_volume(self):
        volume = self.volume_slider.value() / 100
        pygame.mixer.music.set_volume(volume)

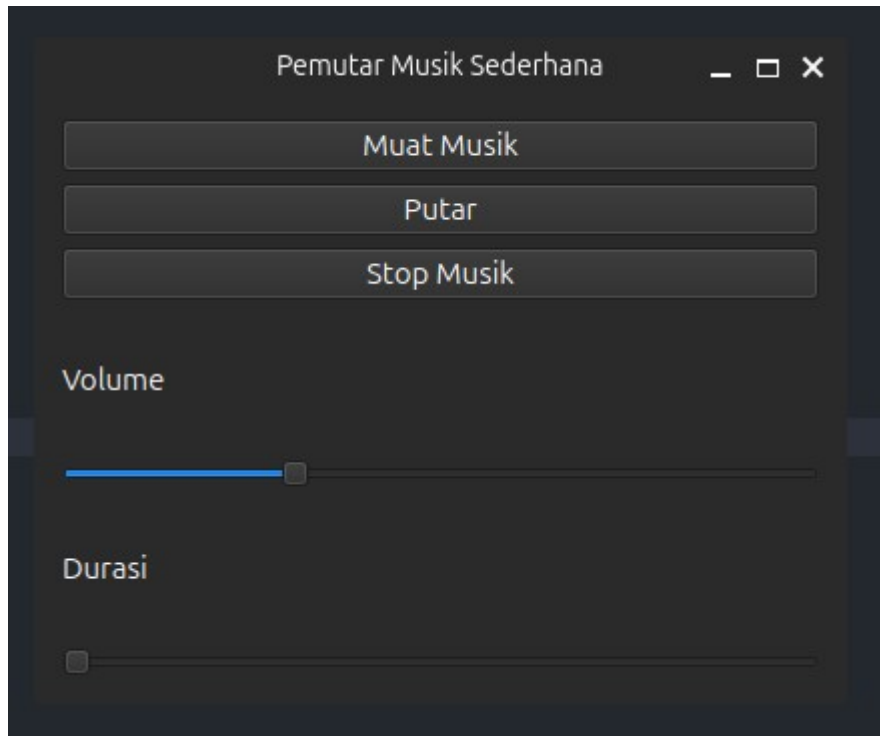
    def update_position(self):
        pos = pygame.mixer.music.get_pos() // 1000 # ms →
detik
        if pos >= 0 and pos <= self.duration:
            self.position_slider.setValue(pos)

    def set_position(self):
        pos = self.position_slider.value()
        pygame.mixer.music.play(start=pos)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    player = MusicPlayer()
    player.show()
    sys.exit(app.exec())

```

Output :



Penjelasan :

Pada project ini kita membuat sebuah aplikasi pemutar musik sederhana dengan antarmuka grafis (GUI) menggunakan **PySide6**, yang menyediakan semua elemen visual seperti jendela, tombol `QPushButton`, dan slider `QSlider`. Untuk fungsionalitas pemutaran audio, program ini mengandalkan library **pygame** yang bertugas untuk memuat, memutar, menghentikan, dan mengatur volume file musik. Sementara itu, library **mutagen** digunakan secara spesifik untuk membaca metadata file audio guna mendapatkan durasi lagu yang akurat. disini mutagen berguna untuk membaca file yang menggunakan ekstensi `.mp3`, `.wav`, dan `.ogg`, yang kemudian dipakai untuk mengatur rentang slider durasi. Agar slider durasi dapat bergerak mengikuti alunan musik, sebuah **QTimer** diatur untuk secara berkala memperbarui posisi visual slider sesuai dengan posisi pemutaran lagu saat ini yang didapat dari `pygame`.

TUGAS :

Source Code :

```
from PySide6.QtWidgets import (  
    QApplication,  
    QVBoxLayout,
```

```

QWidget,
QPushButton,
QFileDialog,
QSlider,
QLabel,
QDial,
)

from PySide6.QtCore import (
    QTimer,
    Qt,
)

import pygame
import sys

class MusicPlayer(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Pemutar Musik Sederhana")
        self.setGeometry(300, 200, 300, 150)

        pygame.mixer.init()

        self.layout = QVBoxLayout()

        # qlabel untuk nama dan nim
        self.label_nama = QLabel("Nama : Rafi Achmad
Arlino\nNIM : 3202416039")
        self.layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.layout.addWidget(self.label_nama)

        self.load_button = QPushButton("Muat Musik")
        self.load_button.clicked.connect(self.load_music)
        self.layout.addWidget(self.load_button)

        self.play_button = QPushButton("Putar")
        self.play_button.clicked.connect(self.play_music)
        self.layout.addWidget(self.play_button)

```

```

self.stop_button = QPushButton("Stop Musik")
self.stop_button.clicked.connect(self.stop_music)
self.layout.addWidget(self.stop_button)

"""self.volume_slider = QSlider(Qt.Horizontal)
self.volume_slider.setRange(0, 100)
self.volume_slider.setValue(30)
self.volume_slider.valueChanged.connect(self.set_volume
)

self.layout.addWidget(QLabel("Volume"))
self.layout.addWidget(self.volume_slider)"""

# dial
self.volume_dial = QDial()
self.volume_dial.setRange(-10, 100)
self.volume_dial.setValue(50)
self.volume_dial.setSingleStep(1)

self.volume_dial.valueChanged.connect(self.set_volume_d
ial)

label_volume = QLabel("Volume")
label_volume.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.layout.addWidget(label_volume)
self.layout.addWidget(self.volume_dial)

# slider durasi
self.posisi_slider = QSlider(Qt.Horizontal)
self.posisi_slider.setRange(0, 0)
self.posisi_slider.sliderReleased.connect(self.slider_m
usik)

self.layout.addWidget(QLabel("Durasi Musik"))
self.layout.addWidget(self.posisi_slider)

# Waktu durasi slider
self.durasi = 0
self.start_pos = 0

# waktu update kontrol musik
self.time = QTimer()

```

```

self.time.setInterval(1000)
self.time.timeout.connect(self.update_letak)
self.setLayout(self.layout)

self.setLayout(self.layout)
self.music_file = None

self.set_volume_dial(self.volume_dial.value())

# definisi slider
def slider_musik(self):
    new_pos = self.posisi_slider.value()
    self.start_pos = new_pos
    pygame.mixer.music.play(start=new_pos)

def update_letak(self):
    pos = pygame.mixer.music.get_pos() // 1000
    if pos >= 0:
        current_time = self.start_pos + pos
        if current_time <= self.durasi:
            self.posisi_slider.setValue(current_time)

def load_music(self):
    file_dialog = QFileDialog(self)
    music_file, _ = file_dialog.getOpenFileName(self, "Pilih
file Musik", "", "Audio Files (*.mp3 *.wav *.ogg)")

    if music_file:
        self.music_file = music_file
        pygame.mixer.music.load(self.music_file)

# membaca durasi
try:
    import mutagen
    from mutagen.mp3 import MP3
    audio = MP3(self.music_file)
    self.durasi = int(audio.info.length)

```



```

        self.posisi_slider.setRange(0, self.durasi)
    except :
        print("Durasi tidak bisa dibaca")

def play_music(self):
    if self.music_file:
        pygame.mixer.music.play()
    else:
        print("Silahkan muat file musik terlebih dahulu!")

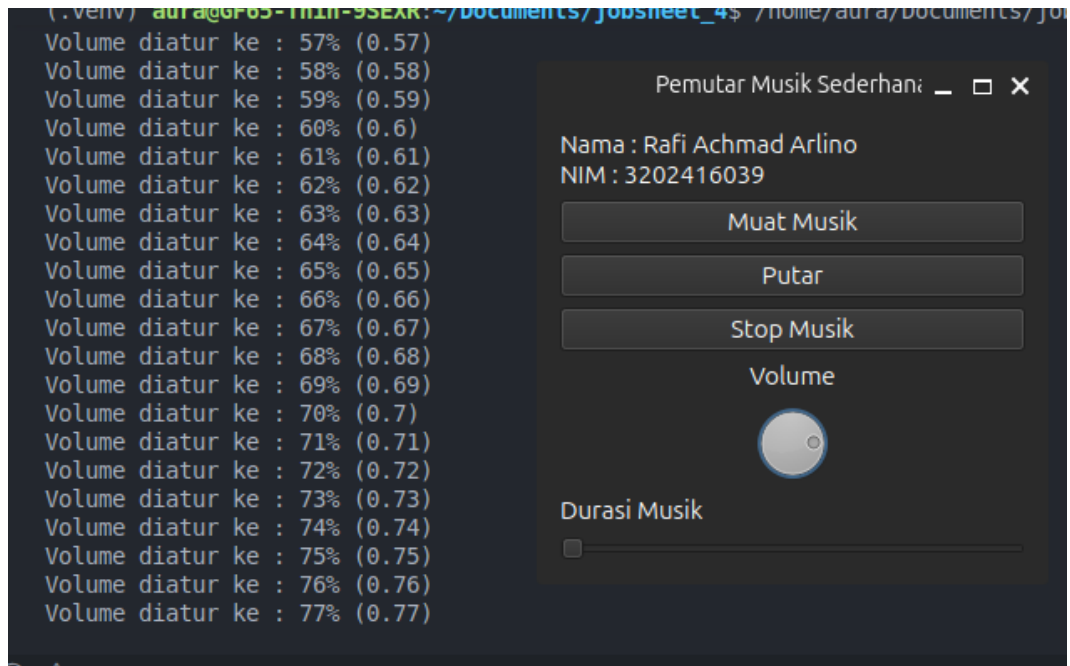
def stop_music(self):
    pygame.mixer.music.stop()

def set_volume_dial(self, value):
    volume_float = value / 100.0
    pygame.mixer.music.set_volume(volume_float)
    print(f"Volume diatur ke : {value}% ({volume_float})")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    player = MusicPlayer()
    player.show()
    sys.exit(app.exec())

```

Output :



Penjelasan :

Kali ini kita mengubah kode project sebuah sebelumnya dengan menambahkan beberapa fitur dengan mengubah slider volume dengan menggunakan dial agar lebih interaktif bagi pengguna dan dilengkapi dengan durasi untuk musik ketika diputar. user dapat dengan bebas mengatur durasi musik cukup dengan menggeser slider dibawah tombol dial.